1. **Watch the Terraform-03 video.**
   Completed.
2. **Execute the script shown in the video.**

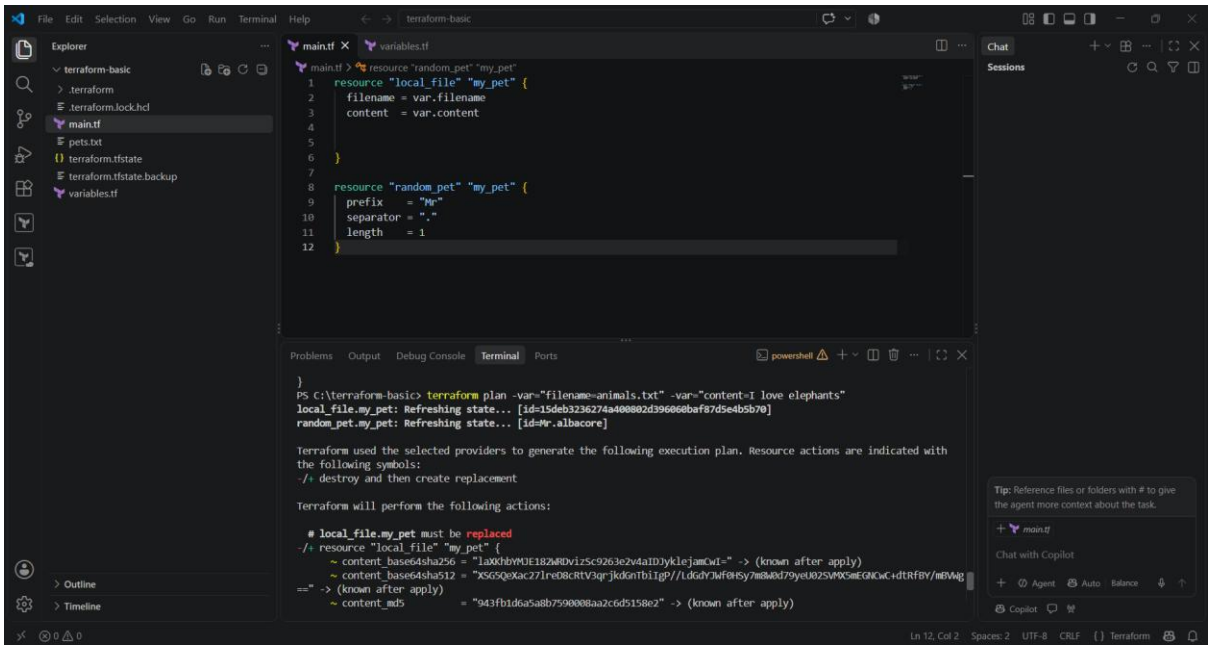
Implementation Steps

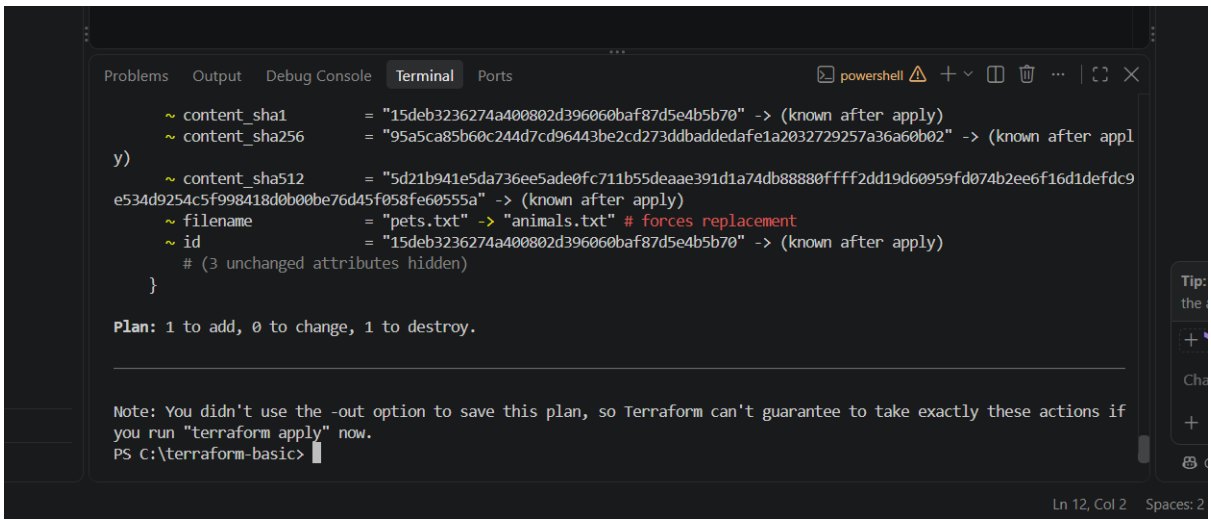
1. Installed Terraform v1.17.0 on Windows and configured the system PATH to access the Terraform executable from PowerShell.
2. Created a Terraform working directory named C:\terraform-basic and prepared the configuration files main.tf, variables.tf, and outputs.tf.
3. Configured the Local provider and created a local_file resource named my_pet to generate the pets.txt file.
4. Initialized the Terraform working directory using terraform init and verified the configuration using terraform validate.
5. Executed terraform plan to review the proposed infrastructure changes before applying them.
6. Used terraform apply to create the resources and verified the generated file and its contents.
7. Added a random_pet resource with the prefix Mr, separator ., and length 1. The generated pet name was used to construct the content of pets.txt.
8. Practiced Terraform input variables through default values, interactive input, the -var command-line option, environment variables using the TF_VAR_ prefix, and .tfvars files.
9. Observed variable precedence by testing different ways of supplying variable values.
10. Used implicit dependencies through resource references and an explicit dependency using depends_on.
11. Configured output values to display the generated pet name after applying the configuration.
12. Inspected Terraform resources and state using terraform state list, terraform state show, and terraform show.
13. Practiced lifecycle rules, including create_before_destroy, prevent_destroy, and ignore_changes. After testing prevent_destroy, removed it from the practice configuration before continuing.
14. Applied the final configuration and verified the output.

Final Verification

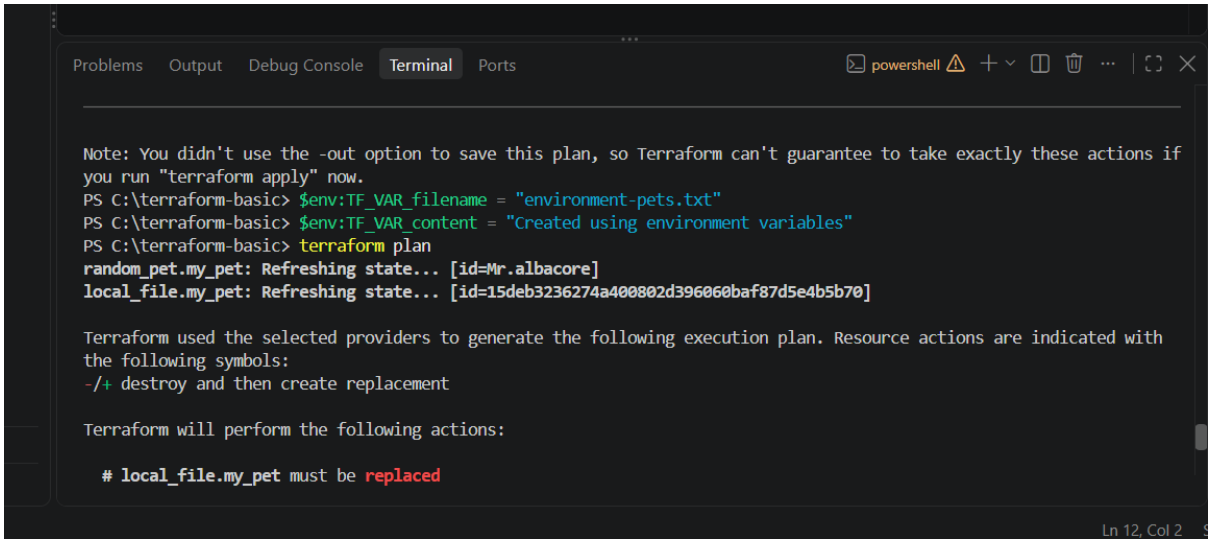
- Terraform version: v1.17.0
- Final output: pet_name = "Mr.albacore"
- Generated file: pets.txt
- File content: My pet name is Mr.albacore

The final resource state and output were verified successfully.





Demonstrate environment variables



```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

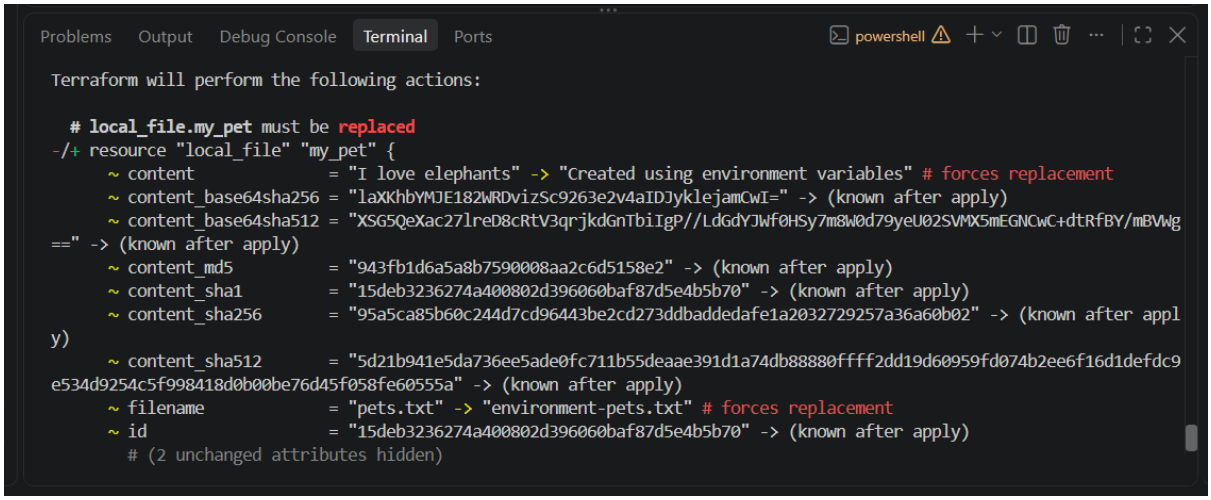
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\terraform-basic> $env:TF_VAR_filename = "environment-pets.txt"
PS C:\terraform-basic> $env:TF_VAR_content = "Created using environment variables"
PS C:\terraform-basic> terraform plan
random_pet.my_pet: Refreshing state... [id=Mr.albacore]
local_file.my_pet: Refreshing state... [id=15deb3236274a400802d396060baf87d5e4b5b70]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my_pet must be replaced

Ln 12, Col 2
```

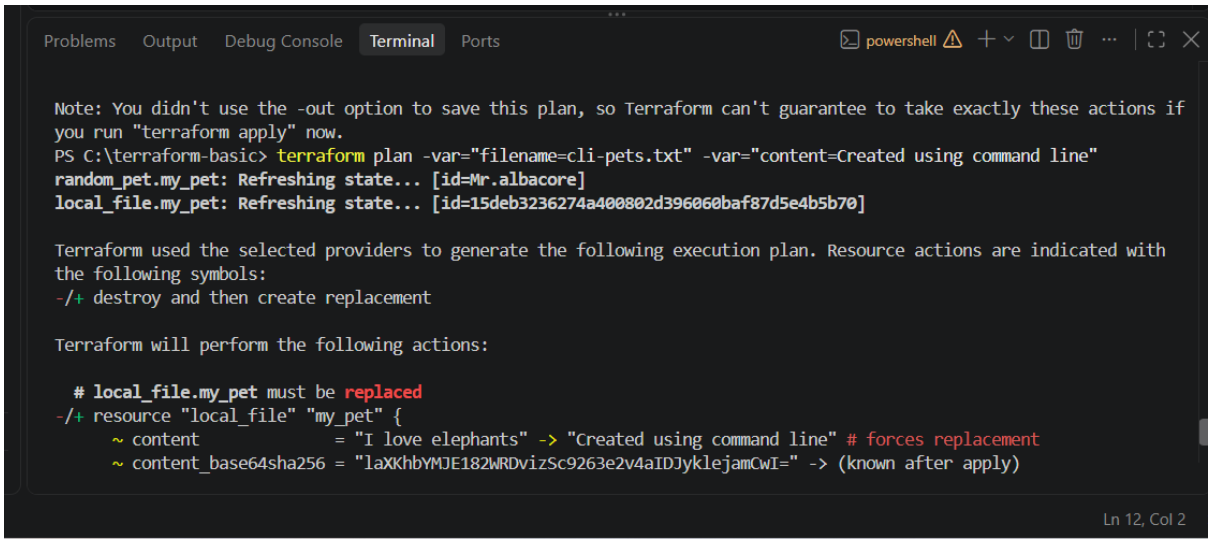


```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "Created using environment variables" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)
      ~ content_base64sha512 = "XSG5QeXac27lreD8cRtV3qrjkdGnTbiIgP//LdGdYJWf0HSy7m8W0d79yeU02SVMX5mEGNCwC+dtRfBY/mBVWg
==" -> (known after apply)
      ~ content_md5          = "943fb1d6a5a8b7590008aa2c6d5158e2" -> (known after apply)
      ~ content_sha1         = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
      ~ content_sha256       = "95a5ca85b60c244d7cd96443be2cd273ddbaddedafe1a2032729257a36a60b02" -> (known after appl
y)
      ~ content_sha512       = "5d21b941e5da736ee5ade0fc711b55deaae391d1a74db88880ffff2dd19d60959fd074b2ee6f16d1defdc9
e534d9254c5f998418d0b00be76d45f058fe60555a" -> (known after apply)
      ~ filename             = "pets.txt" -> "environment-pets.txt" # forces replacement
      ~ id                   = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
        # (2 unchanged attributes hidden)
```

Demonstrate variable precedence



```
Problems  Output  Debug Console  Terminal  Ports                                    powershell

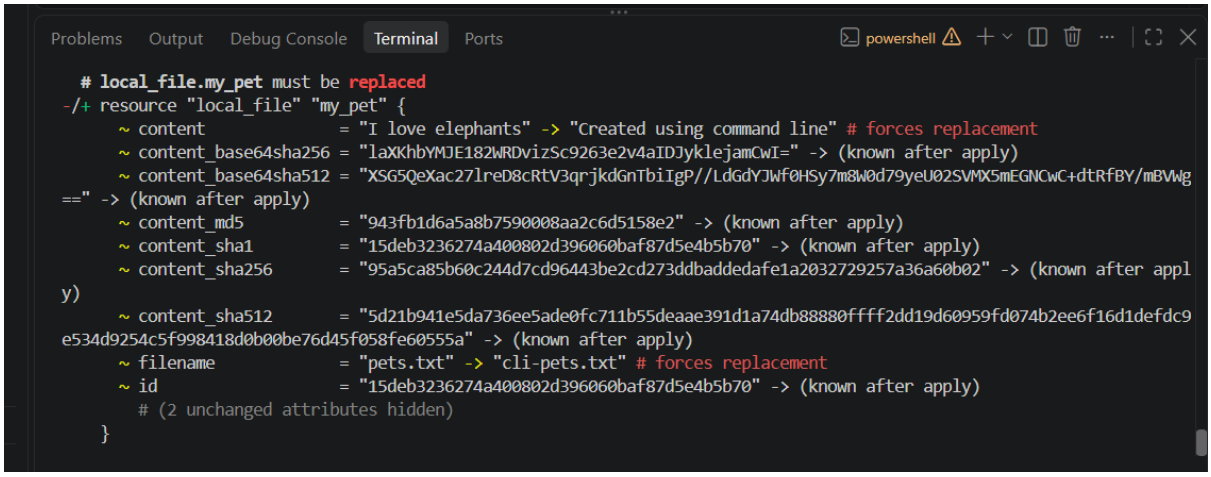
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\terraform-basic> terraform plan -var="filename=cli-pets.txt" -var="content=Created using command line"
random_pet.my_pet: Refreshing state... [id=Mr.albacore]
local_file.my_pet: Refreshing state... [id=15deb3236274a400802d396060baf87d5e4b5b70]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
-/+ destroy and then create replacement

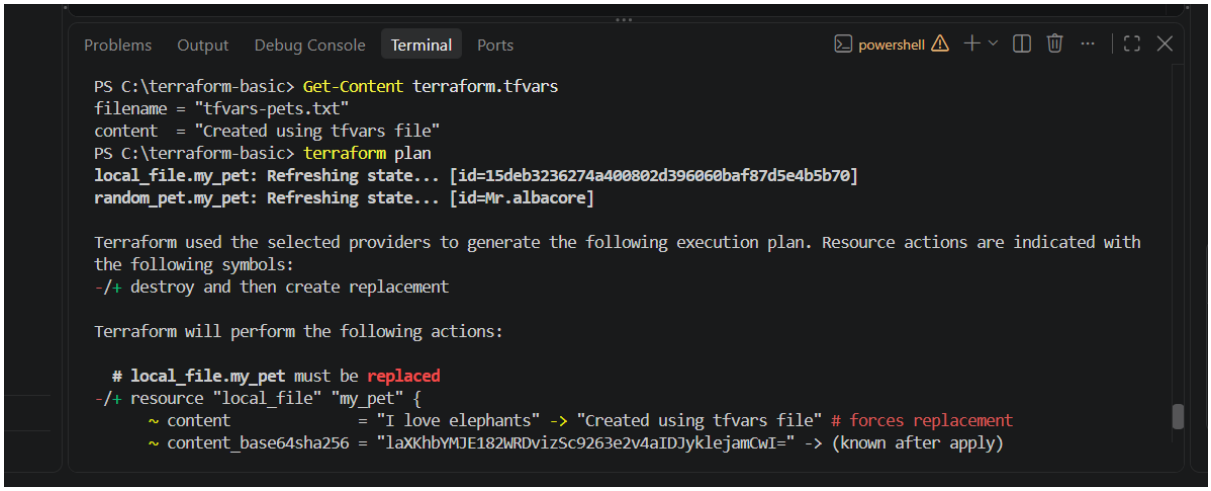
Terraform will perform the following actions:

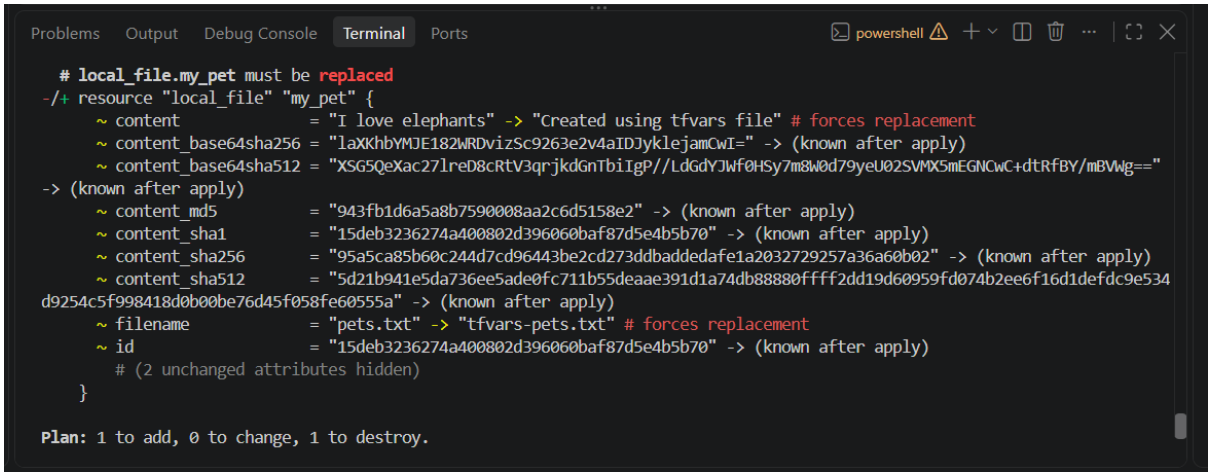
  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "Created using command line" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)

Ln 12, Col 2
```



Demonstrate variables using a .tfvars file





Demonstrate a custom-named .tfvars file

```
PS C:\terraform-basic> Get-Content training.tfvars
filename = "training-pets.txt"
content  = "Created using custom tfvars file"
PS C:\terraform-basic> terraform plan -var-file="training.tfvars"
random_pet.my_pet: Refreshing state... [id=Mr.albacore]
local_file.my_pet: Refreshing state... [id=15deb3236274a400802d396060baf87d5e4b5b70]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

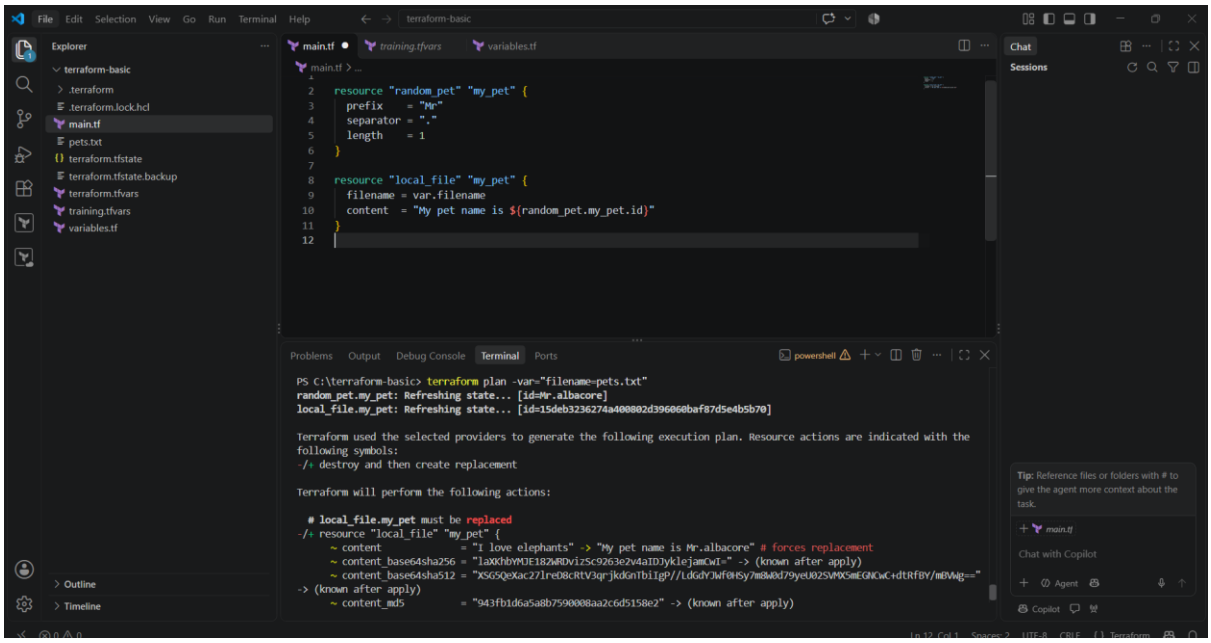
  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "Created using custom tfvars file" # forces replacement
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "Created using custom tfvars file" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)
      ~ content_base64sha512 = "XSG5QeXac27lreD8cRtV3qrjkdGnTbiIgP//LdGdYJWf0HSy7m8W0d79yeU02SVMX5mEGNCwC+dtRfBY/mBVWg=="
-> (known after apply)
      ~ content_md5          = "943fb1d6a5a8b7590008aa2c6d5158e2" -> (known after apply)
      ~ content_sha1         = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
      ~ content_sha256       = "95a5ca85b60c244d7cd96443be2cd273ddbaddedafe1a2032729257a36a60b02" -> (known after apply)
      ~ content_sha512       = "5d21b941e5da736ee5ade0fc711b55deaae391d1a74db88880ffff2dd19d60959fd074b2ee6f16d1defdc9e534
d9254c5f998418d0b00be76d45f058fe60555a" -> (known after apply)
      ~ filename             = "pets.txt" -> "training-pets.txt" # forces replacement
      ~ id                   = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

Demonstrate resource dependencies

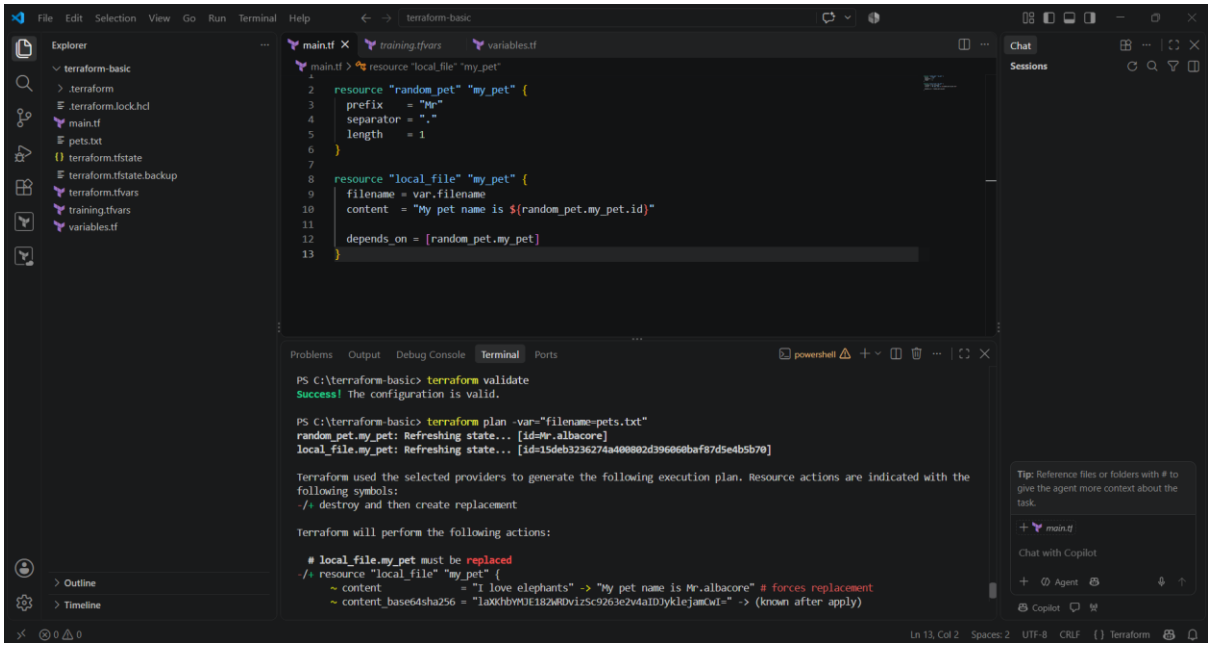


```
Problems  Output  Debug Console  Terminal  Ports                                   powershell

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "My pet name is Mr.albacore" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)
      ~ content_base64sha512 = "XSG5QeXac27lreD8cRtV3qrjkdGnTbiIgP//LdGdYJWf0HSy7m8W0d79yeU02SVMX5mEGNCwC+dtRfBY/mBVWg=="
-> (known after apply)
      ~ content_md5          = "943fb1d6a5a8b7590008aa2c6d5158e2" -> (known after apply)
      ~ content_sha1         = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
      ~ content_sha256       = "95a5ca85b60c244d7cd96443be2cd273ddbaddedafe1a2032729257a36a60b02" -> (known after apply)
      ~ content_sha512       = "5d21b941e5da736ee5ade0fc711b55deaae391d1a74db88880ffff2dd19d60959fd074b2ee6f16d1defdc9e534
d9254c5f998418d0b00be76d45f058fe60555a" -> (known after apply)
      ~ id                   = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
        # (3 unchanged attributes hidden)
    }
```

Demonstrate output variables

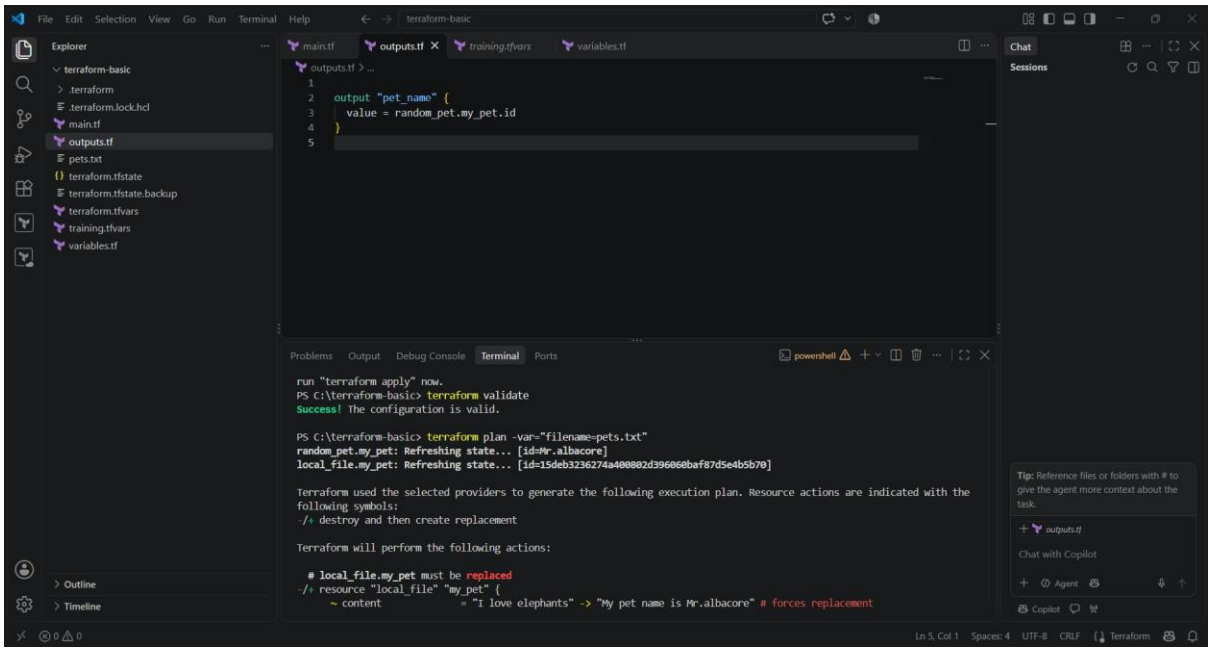


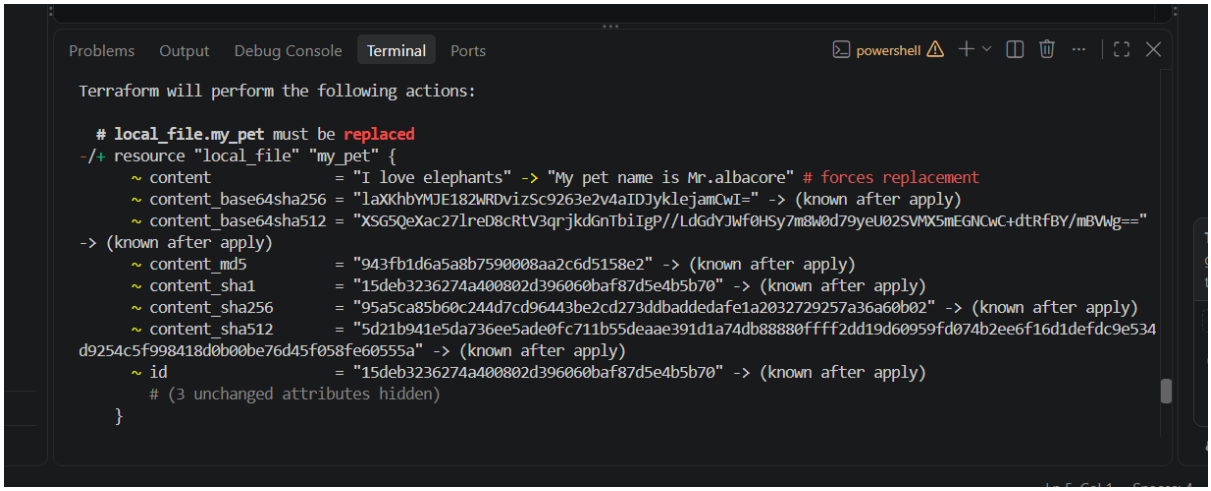

```
Problems  Output  Debug Console  Terminal  Ports                                   powershell

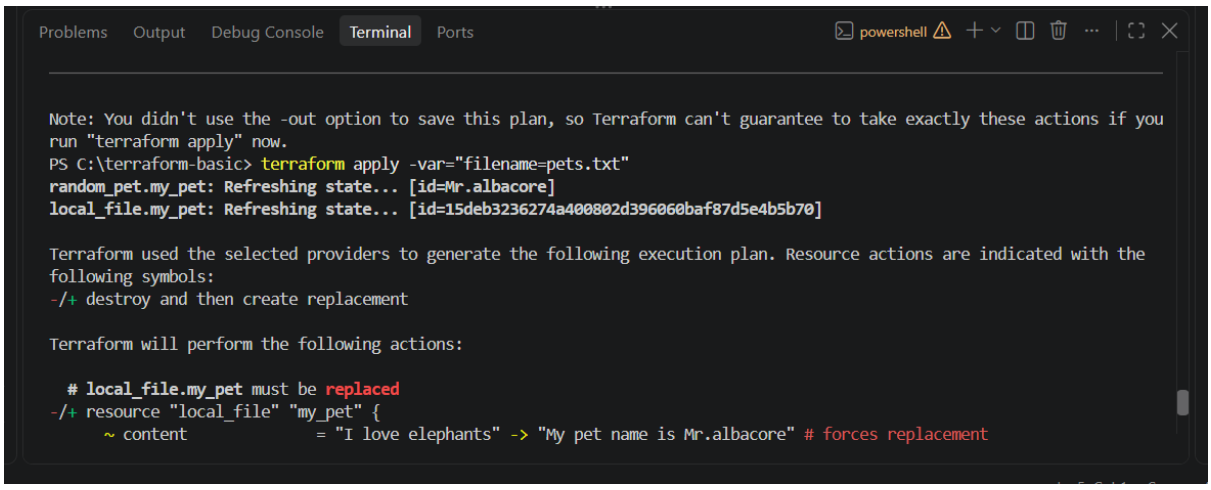
  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "My pet name is Mr.albacore" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)
      ~ content_base64sha512 = "XSG5QeXac27lreD8cRtV3qrjkdGnTbiIgP//LdGdYJWf0HSy7m8W0d79yeU02SVMX5mEGNCwC+dtRfBY/mBVWg=="
-> (known after apply)
      ~ content_md5          = "943fb1d6a5a8b7590008aa2c6d5158e2" -> (known after apply)
      ~ content_sha1         = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
      ~ content_sha256       = "95a5ca85b60c244d7cd96443be2cd273ddbaddedafe1a2032729257a36a60b02" -> (known after apply)
      ~ content_sha512       = "5d21b941e5da736ee5ade0fc711b55deaae391d1a74db88880ffff2dd19d60959fd074b2ee6f16d1defdc9e534
d9254c5f998418d0b00be76d45f058fe60555a" -> (known after apply)
      ~ id                   = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
        # (3 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

Demonstrate output variables







```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "I love elephants" -> "My pet name is Mr.albacore" # forces replacement
      ~ content_base64sha256 = "laXKhbYMJE182WRDvizSc9263e2v4aIDJyklejamCwI=" -> (known after apply)
      ~ content_base64sha512 = "XSG5QeXac27lreD8cRtV3qrjkdGnTbiIgP//LdGdYJWf0HSy7m8W0d79yeU02SVMX5mEGNCwC+dtRfBY/mBVWg==" 
-> (known after apply)
      ~ content_md5          = "943fb1d6a5a8b7590008aa2c6d5158e2" -> (known after apply)
      ~ content_sha1         = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
      ~ content_sha256       = "95a5ca85b60c244d7cd96443be2cd273ddbaddedafe1a2032729257a36a60b02" -> (known after apply)
      ~ content_sha512       = "5d21b941e5da736ee5ade0fc711b55deaae391d1a74db88880ffff2dd19d60959fd074b2ee6f16d1defdc9e534
d9254c5f998418d0b00be76d45f058fe60555a" -> (known after apply)
      ~ id                   = "15deb3236274a400802d396060baf87d5e4b5b70" -> (known after apply)
        # (3 unchanged attributes hidden)
    }
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  + pet_name = "Mr.albacore"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=15deb3236274a400802d396060baf87d5e4b5b70]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=0fc87516e209b074f78ada148c40ff6358cc0563]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

```
local_file.my_pet: Creation complete after 0s [id=0fc87516e209b074f78ada148c40ff6358cc0563]

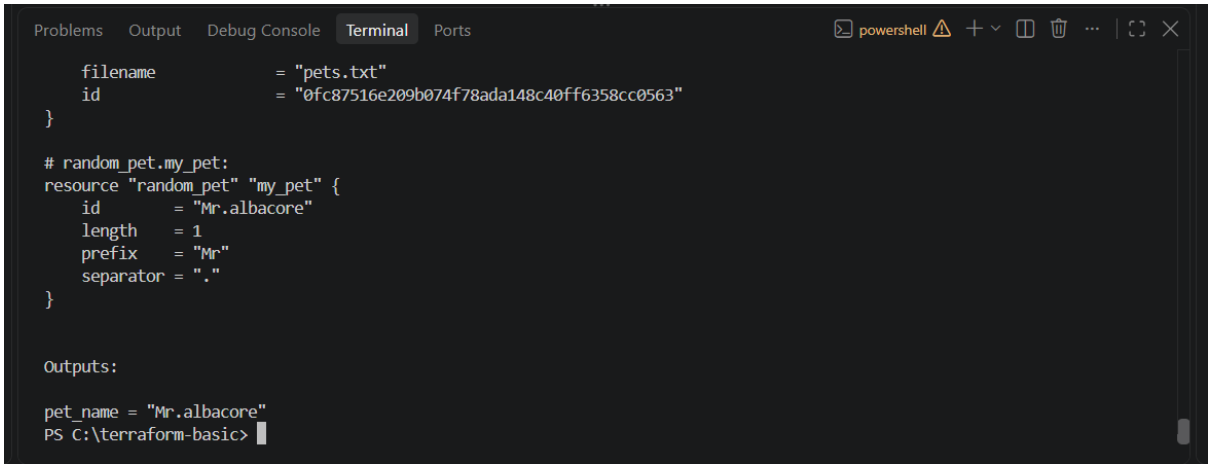
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

pet_name = "Mr.albacore"
PS C:\terraform-basic> terraform output
pet_name = "Mr.albacore"
PS C:\terraform-basic> terraform output pet_name
"Mr.albacore"
PS C:\terraform-basic> Get-Content pets.txt
My pet name is Mr.albacore
PS C:\terraform-basic>
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform-basic> terraform show
# local_file.my_pet:
resource "local_file" "my_pet" {
    content              = "My pet name is Mr.albacore"
    content_base64sha256 = "5Ixy8DQByLxbNGdd9xS6a+mZF/5hm7GB7XEyANNtOzc="
    content_base64sha512 = "rBhByb9uCN/TeiaiOokHmcxVnpuAWtyb3zQbGe8ZlslO6E6GB5OkKn93Xl/45VDrtSEASf7VYmZv8XOpr26HnA=="
    content_md5          = "ba45e5090344bd5e2452f4cb2278c1bb"
    content_sha1         = "0fc87516e209b074f78ada148c40ff6358cc0563"
    content_sha256       = "e48c72f03401c8bc5b34675df714ba6be99917fe619bb181ed713200d36d3b37"
    content_sha512       = "ac1841c9bf6e08dfd37a26a23a890799cc559e9b805adc9bdf341b19ef1996c94ee84e860793a42a7f775e5ff8e550
ebb5210049fed562666ff173a9af6e879c"
    directory_permission = "0777"
    file_permission      = "0777"
    filename             = "pets.txt"
    id                   = "0fc87516e209b074f78ada148c40ff6358cc0563"
}
```



3. **Integrate Terraform in Jenkins using the Terraform plugin.**

Implementation Steps

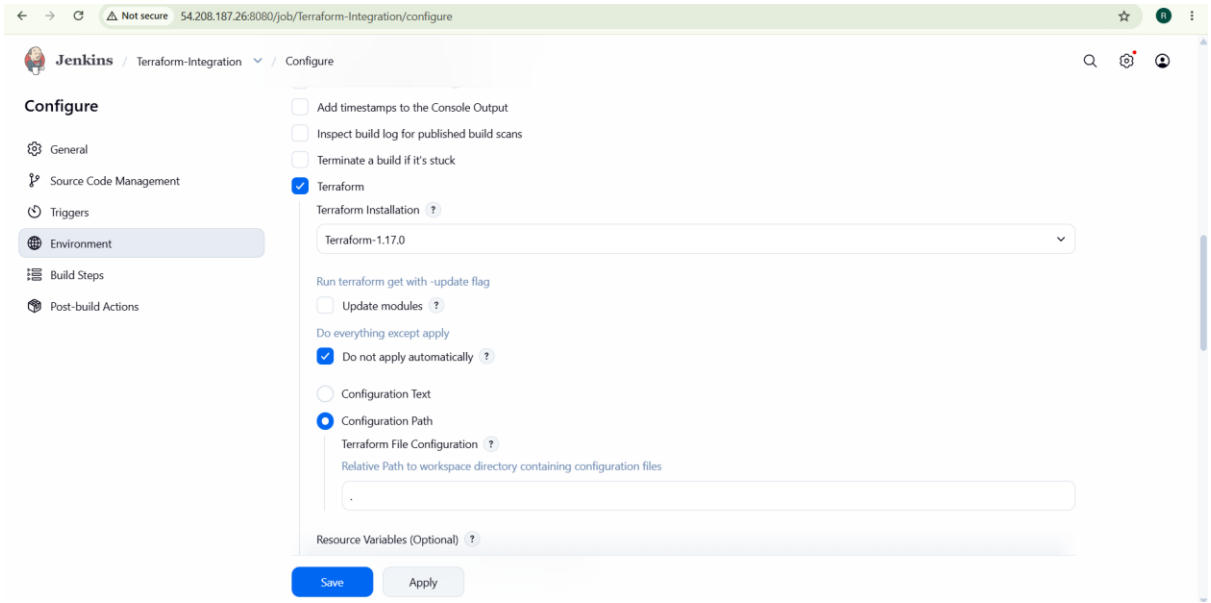
1. Accessed the Jenkins dashboard and opened **Manage Jenkins → Plugins**.
2. Installed the Terraform plugin required for the integration.
3. Opened **Manage Jenkins → Tools** and configured the Terraform installation with the name Terraform-1.17.0.
4. Used /usr/bin as the Terraform installation directory and left automatic installation unchecked.
5. Verified that Terraform was installed on the Jenkins EC2 instance and was accessible at /usr/bin/terraform.
6. Created a Freestyle Jenkins job named Terraform-Integration.
7. Selected the configured Terraform installation, Terraform-1.17.0, in the Terraform build environment.
8. Configured the Terraform plugin's configuration path as . to refer to the job workspace.
9. Enabled **Do not apply automatically** to prevent the plugin from automatically applying the Terraform configuration.
10. Added an Execute shell build step that created a main.tf file in the Jenkins workspace using a shell heredoc.

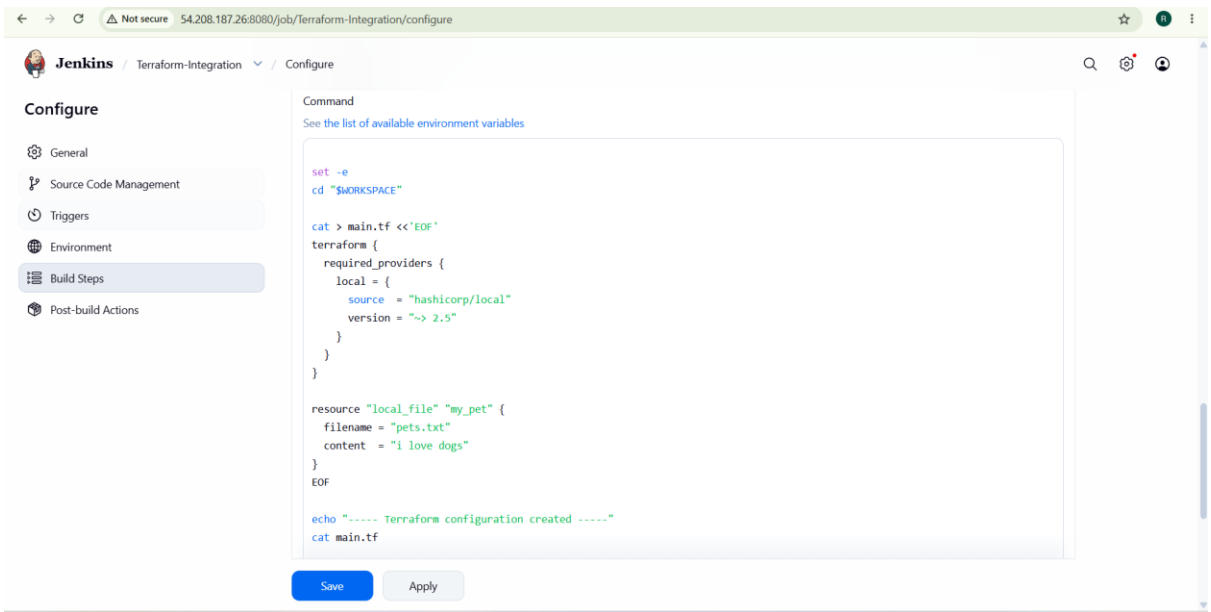
11. Configured the Terraform file to use the Local provider and a local_file resource named my_pet, with pets.txt as the filename and i love dogs as the content.
12. Printed the generated Terraform configuration in the Jenkins build log for verification.
13. Saved the job and executed the build.

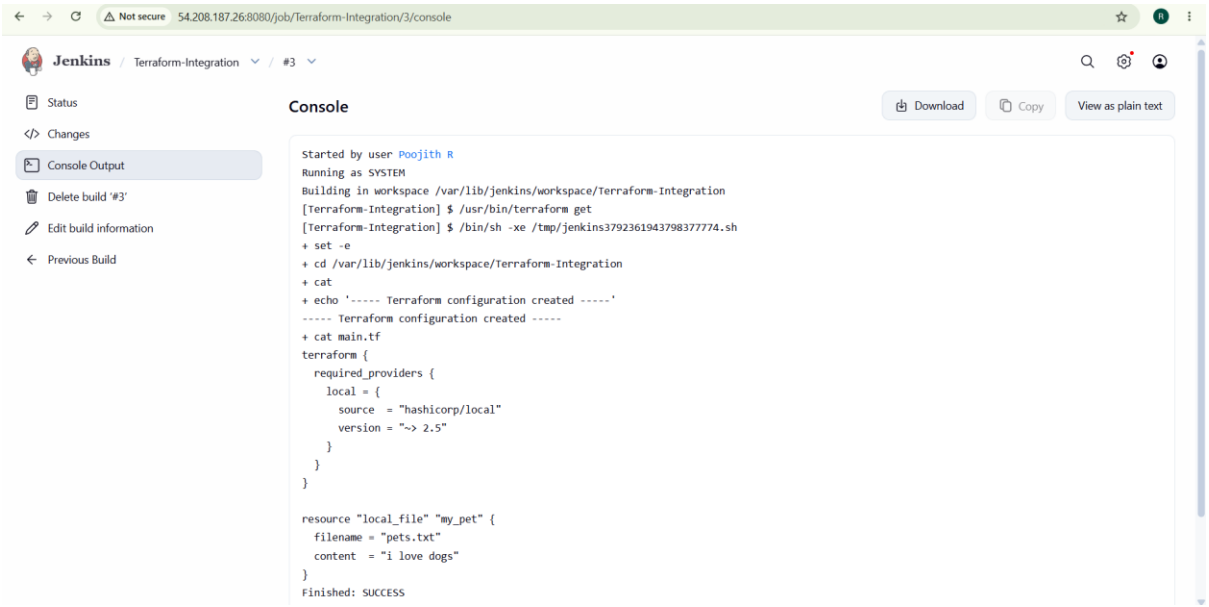
Final Verification

The Jenkins console output showed the Terraform plugin executing /usr/bin/terraform get, displaying the generated main.tf configuration, and finishing with:

Finished: SUCCESS







4. **Create one Jenkins job using Maven Project for the code below with two stages:**
    - **Stage 1: Git clone**
    - **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

Implementation Steps

1. Configured Maven under **Manage Jenkins → Tools** using the installation name Maven-3.9.4. The configured automatic installation selected Maven 3.10.0.
2. Installed the Java 21 development package on the Jenkins EC2 instance because the existing Corretto installation did not initially contain javac.
3. Verified the Java compiler using:

javac -version

The output confirmed Java compiler version 21.0.13.

4. Added the installed JDK to Jenkins under **Manage Jenkins → Tools** with the name JDK21 and the Java home directory /usr/lib/jvm/java-21-amazon-corretto.x86_64.
5. Created a Pipeline job named Java-Working-App.

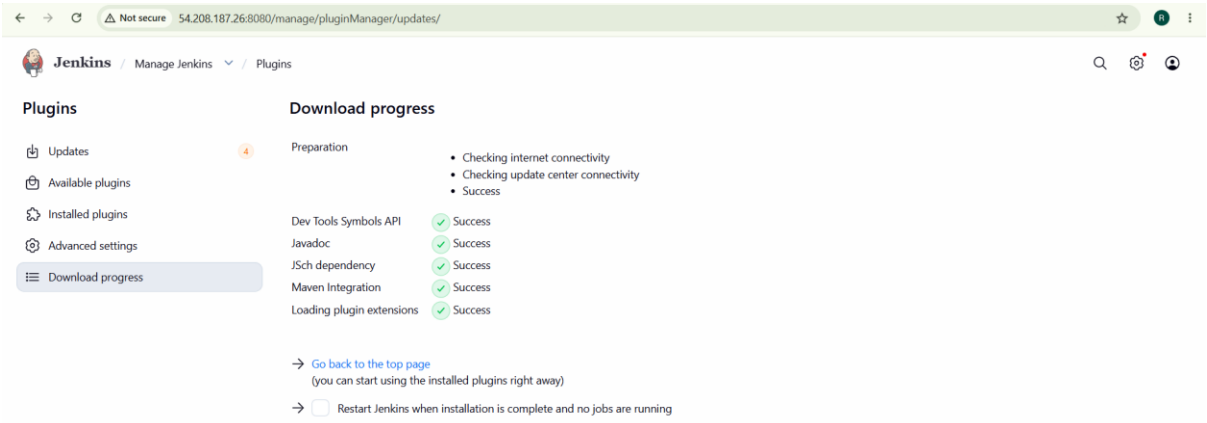
6. Selected **Pipeline script** as the Pipeline definition.
7. Configured the Pipeline to use the tools JDK21 and Maven-3.9.4.
8. Added a Git Clone stage to retrieve the repository https://github.com/betawins/java-Working-app.git from the main branch.
9. Added a Maven Compilation stage that executed java -version, mvn -version, and mvn clean compile.
10. Saved the Pipeline and executed the job using **Build Now**.
11. Inspected the Console Output to verify the Git checkout and Maven execution.

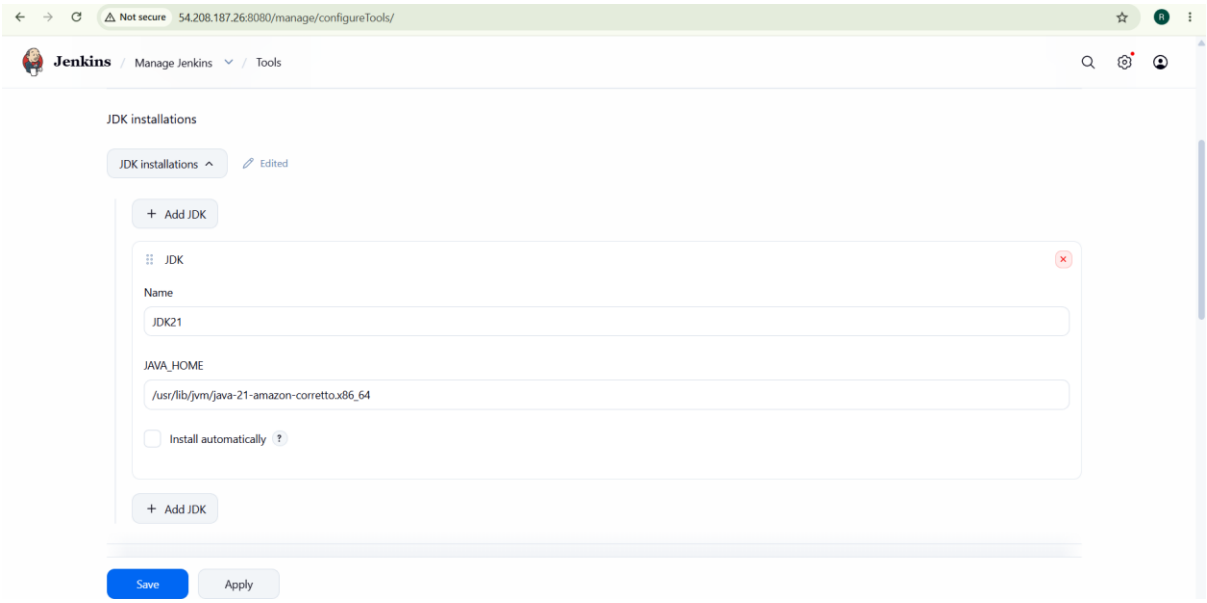
Final Verification

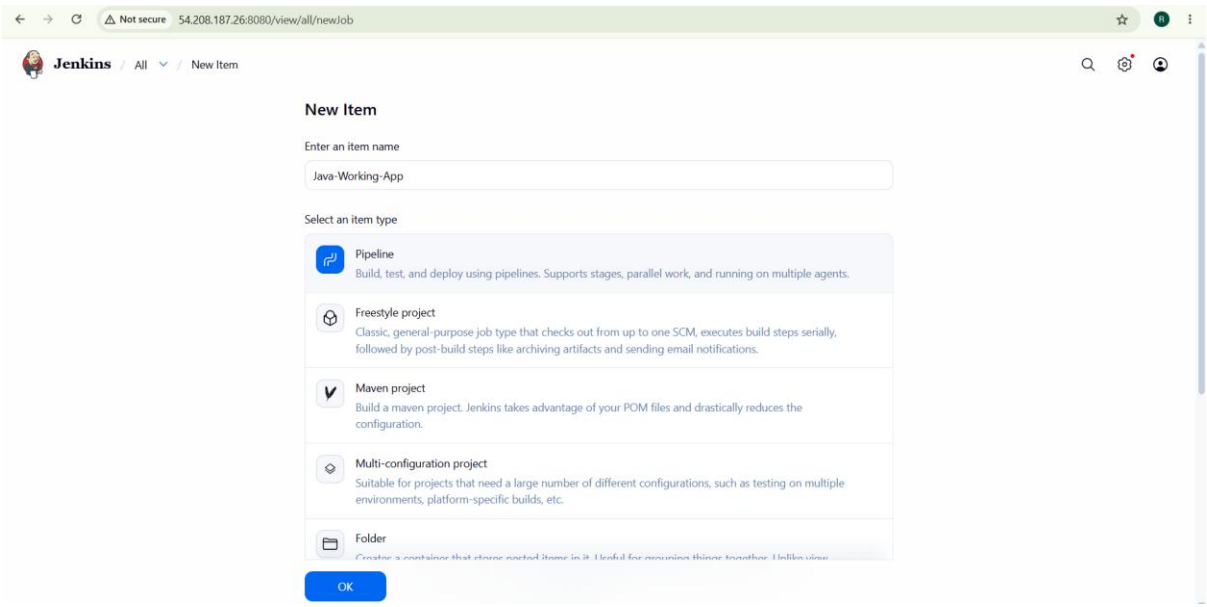
The Pipeline completed with:

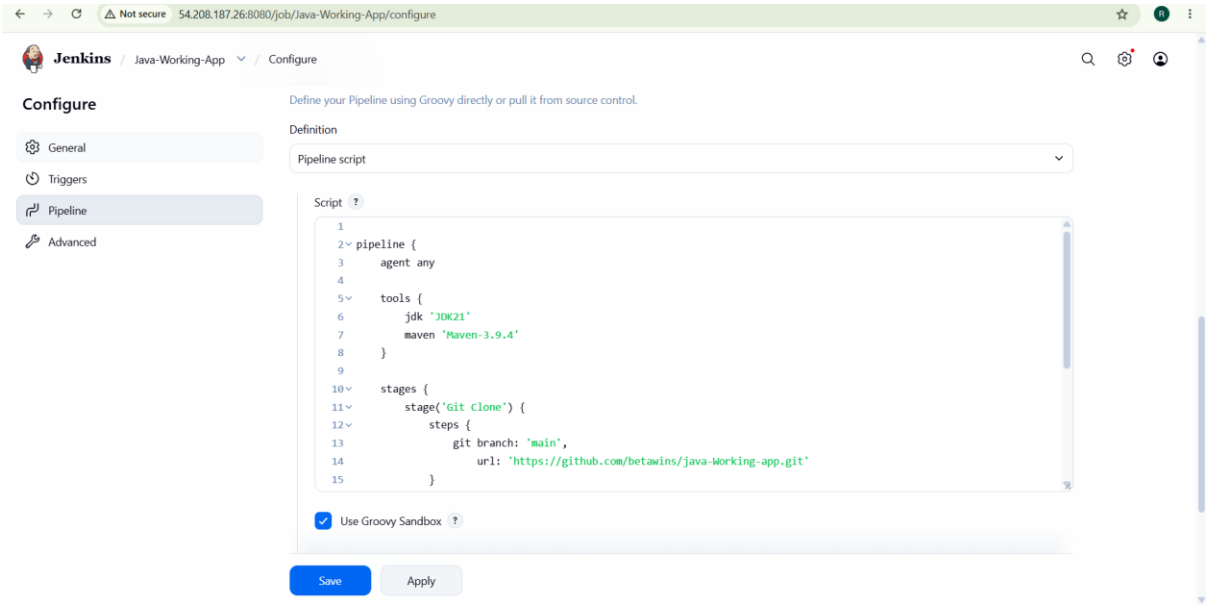
- Successful Git checkout from the main branch.
- Java and Maven version information displayed.
- Maven result: BUILD SUCCESS
- Jenkins result: Finished: SUCCESS

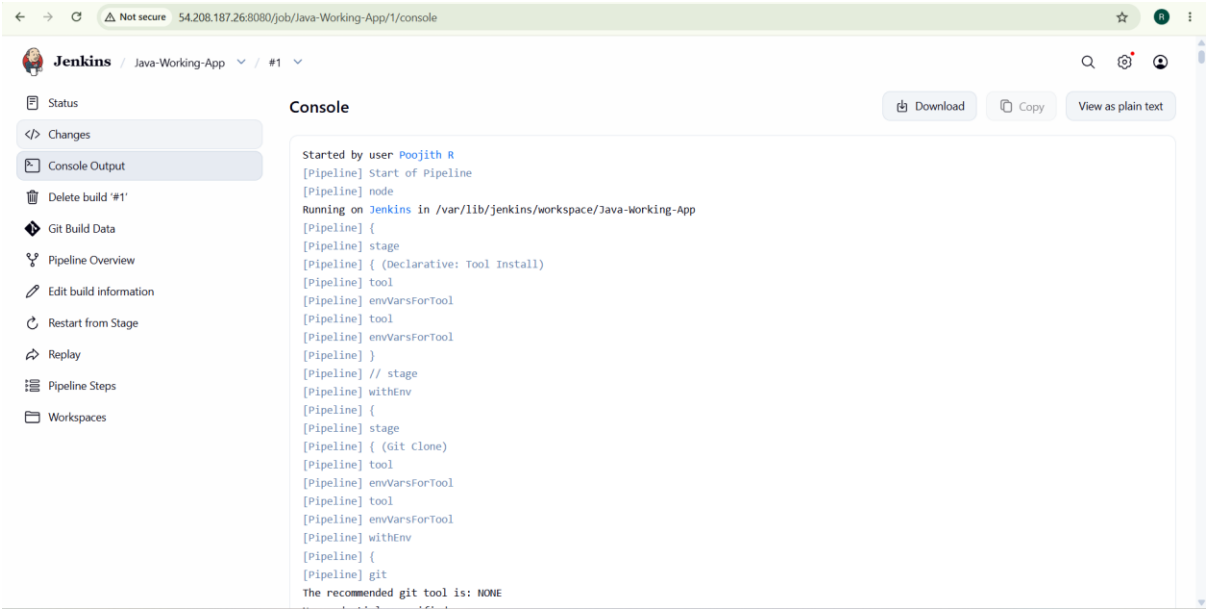
The output also reported No sources to compile, indicating that Maven did not find source files to compile in its configured source directory during that run.

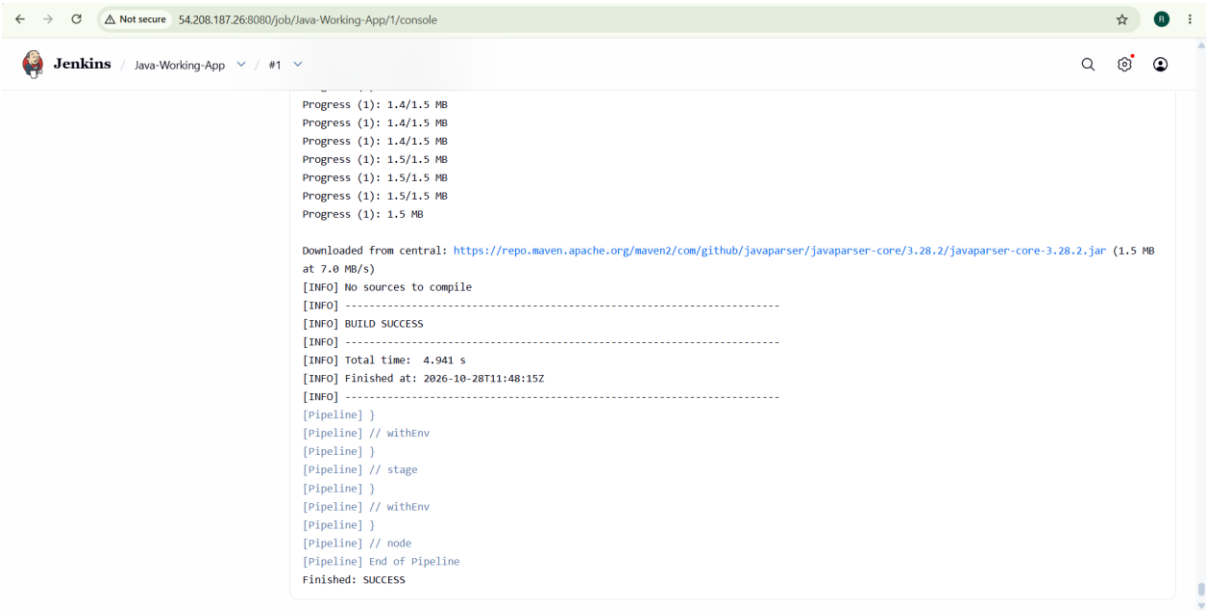



Not secure 54.208.187.26:8080/manage/configureTools/
Jenkins / Manage Jenkins / Tools
JDK installations
JDK installations
Edited
+ Add JDK
JDK
Name
JDK21
JAVA_HOME
/usr/lib/jvm/java-21-amazon-corretto.x86_64
Install automatically
+ Add JDK
Save
Apply


Not secure 54.208.187.26:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Java-Working-App
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
OK


Not secure 54.208.187.26:8080/job/Java-Working-App/configure
Jenkins / Java-Working-App / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
tools {
jdk 'JDK21'
maven 'Maven-3.9.4'
}
stages {
stage('Git Clone') {
steps {
git branch: 'main',
url: 'https://github.com/betawins/java-Working-app.git'
}
Use Groovy Sandbox
Save
Apply





5. **Use the same code and create a parameterized job in Jenkins with:**
    - **Stage 1: Git clone**
    - **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

Implementation Steps

1. Created a separate Pipeline job named Java-Working-App-Parameterized.
2. Enabled **This project is parameterized** under the General section.

3. Added a Choice Parameter named ENVIRONMENT with the choices dev, qa, and prod.
4. Added a Boolean Parameter named DEPLOY, with its default value set to false.
5. Configured the Pipeline to use the tools JDK21 and Maven-3.9.4.
6. Added a Git Clone stage to retrieve https://github.com/betawins/java-Working-app.git from the main branch.
7. Added a Maven Compilation stage that executed mvn clean compile.
8. Added a Deployment Decision stage that read the selected environment through params.ENVIRONMENT and checked the Boolean value through params.DEPLOY.
9. Configured the Pipeline to print a deployment-enabled message when DEPLOY was true and a deployment-skipped message when it was false.
10. Saved the Pipeline and selected **Build with Parameters** to execute the job.
11. Selected dev for ENVIRONMENT and left DEPLOY unchecked.
12. Inspected the Console Output to verify that the selected environment was read correctly and the deployment decision was handled as expected.

Final Verification

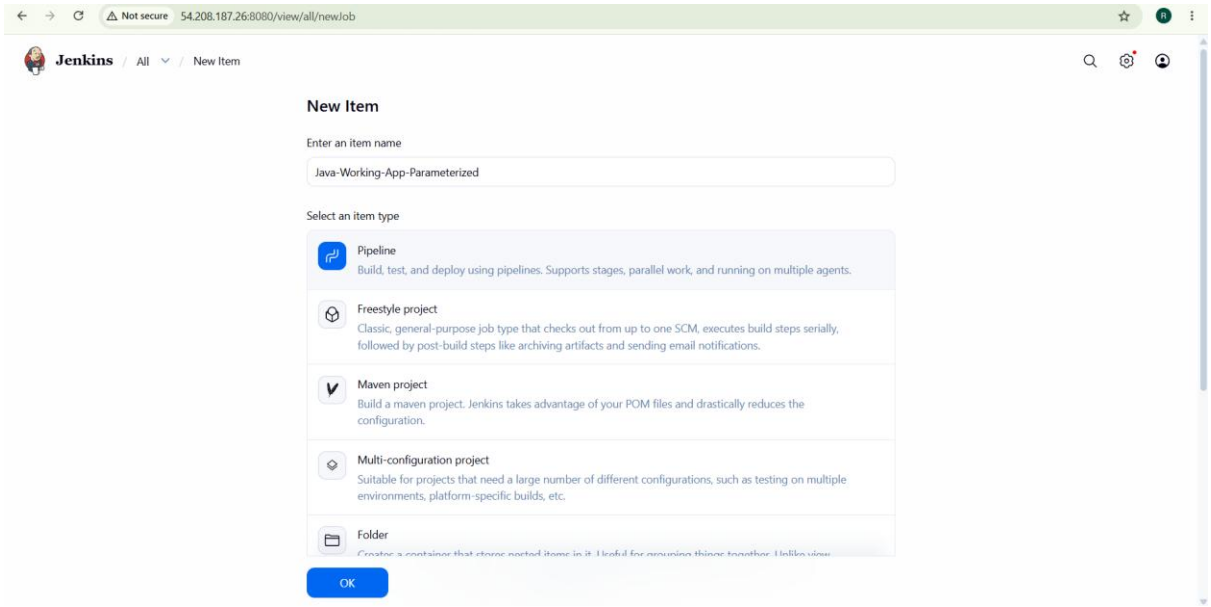
The Console Output showed:

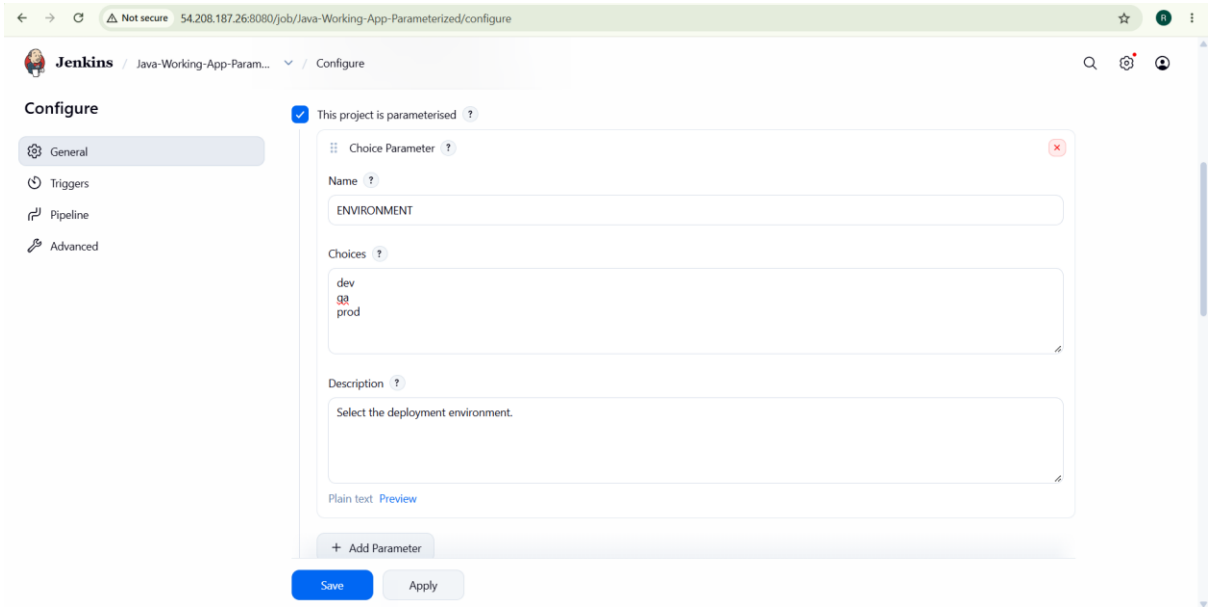
Selected environment: dev

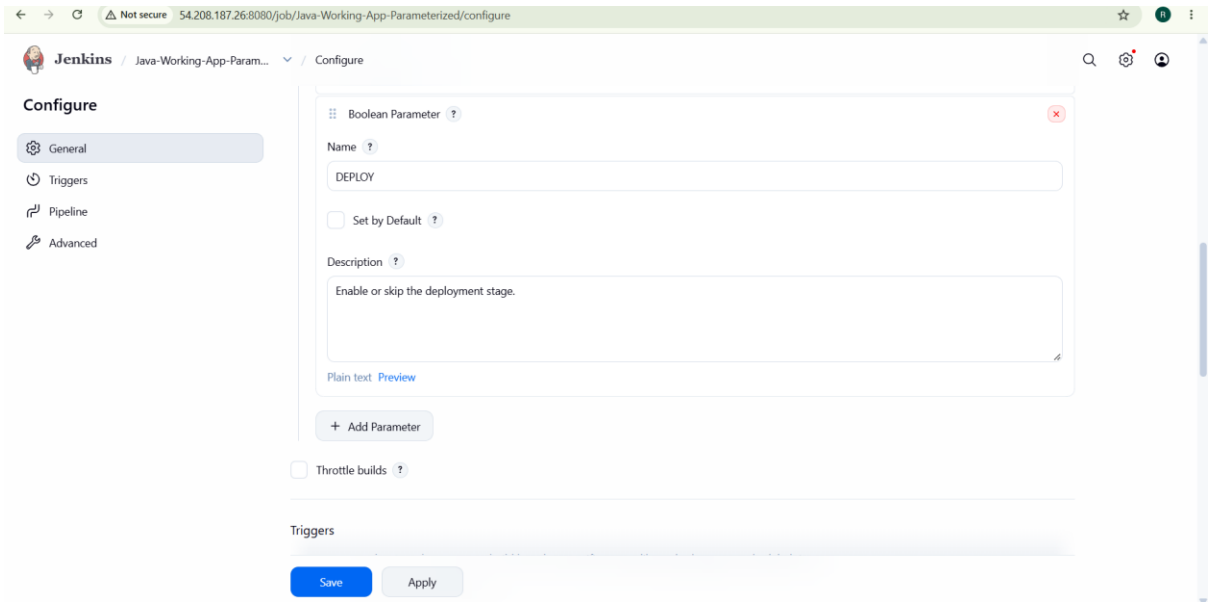
Deployment skipped because DEPLOY is false

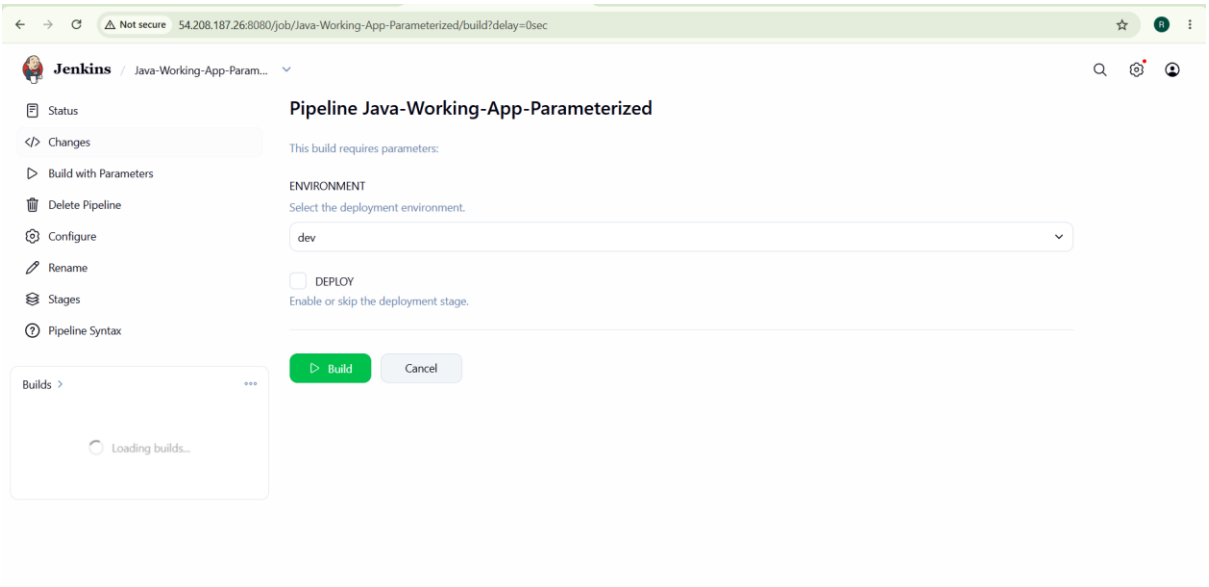
Finished: SUCCESS

The test verified the parameterized Pipeline and its false branch.

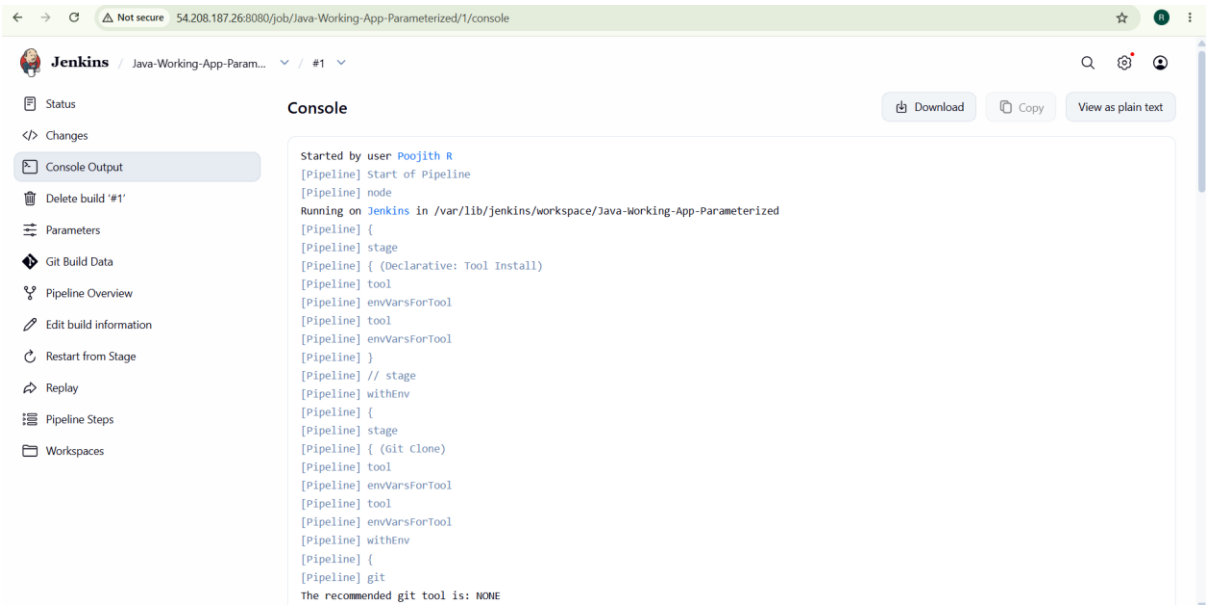

Not secure 54.208.187.26:8080/view/all/newJob
Jenkins / All / New Item
New Item
Enter an item name
Java-Working-App-Parameterized
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
OK

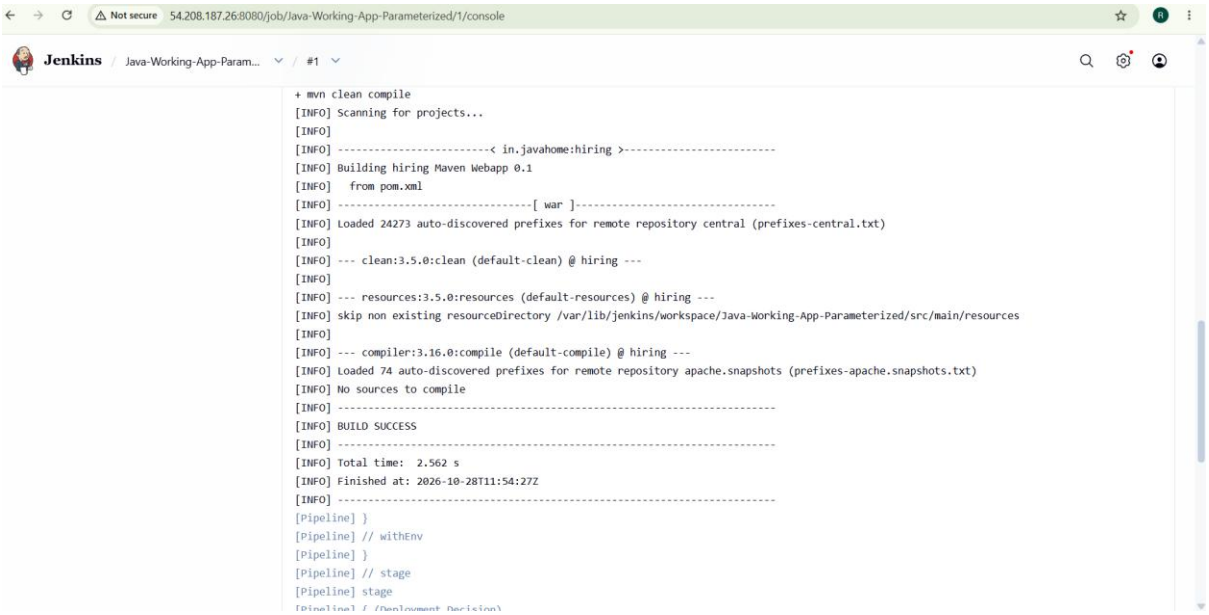

Not secure 54.208.187.26:8080/job/Java-Working-App-Parameterized/configure
Jenkins / Java-Working-App-Param... / Configure
Configure
General
Triggers
Pipeline
Advanced
This project is parameterised
Choice Parameter
Name
ENVIRONMENT
Choices
dev
qa
prod
Description
Select the deployment environment.
Plain text Preview
Add Parameter
Save
Apply

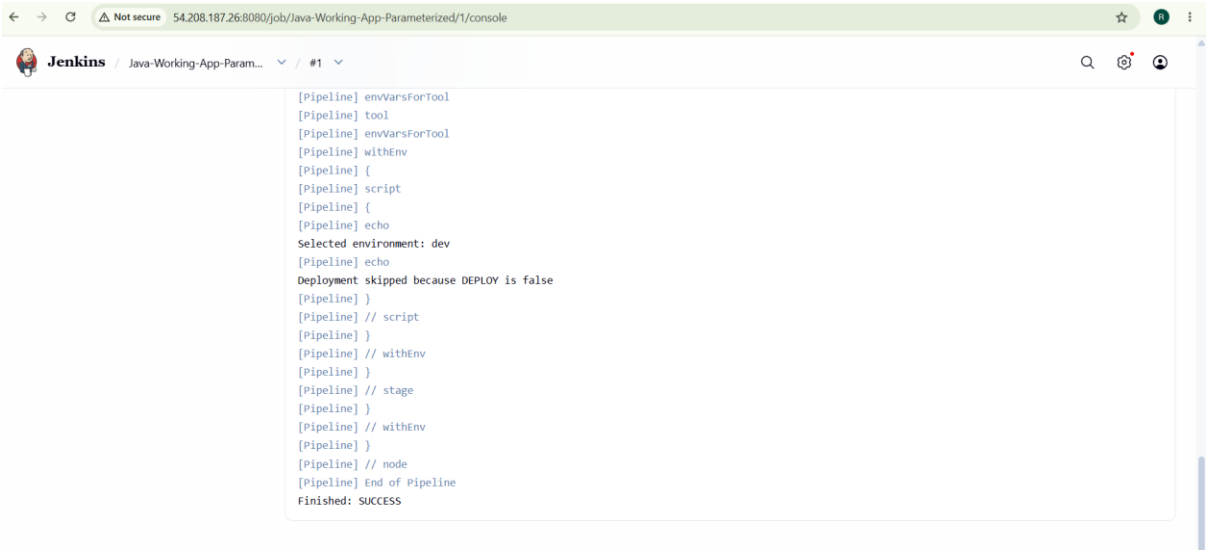

Not secure 54.208.187.26:8080/job/Java-Working-App-Parameterized/configure
Jenkins / Java-Working-App-Param... / Configure
Configure
General
Triggers
Pipeline
Advanced
Boolean Parameter
Name
DEPLOY
Set by Default
Description
Enable or skip the deployment stage.
Plain text Preview
Add Parameter
Throttle builds
Triggers
Save
Apply

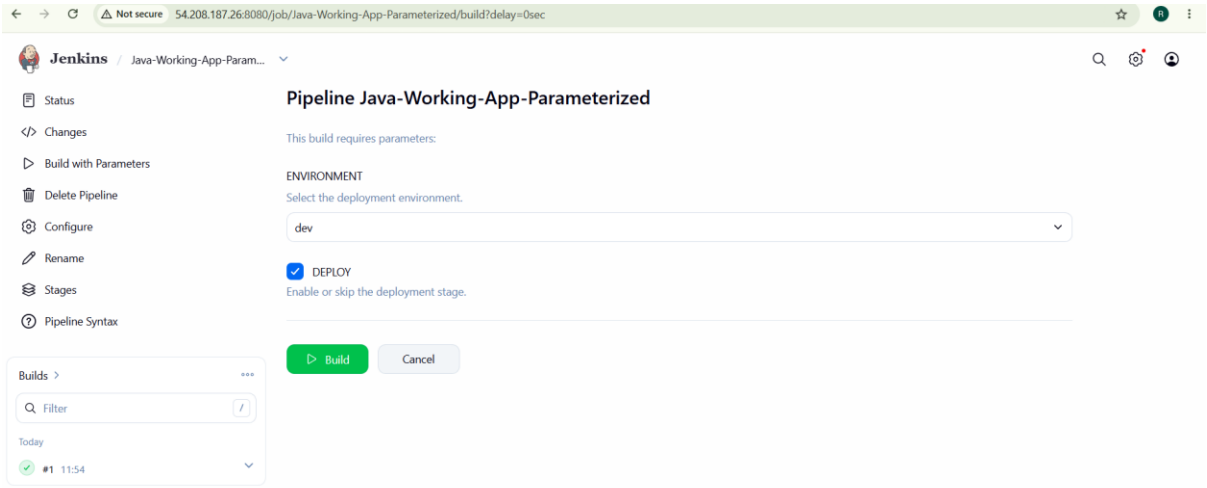


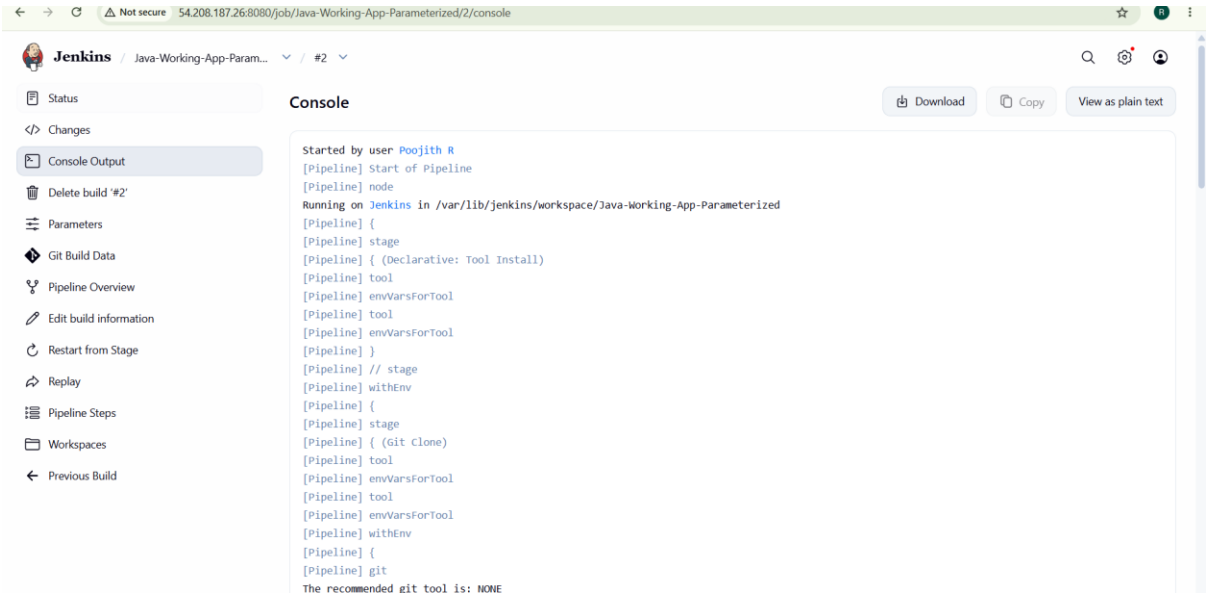
Jenkins 2.580.1

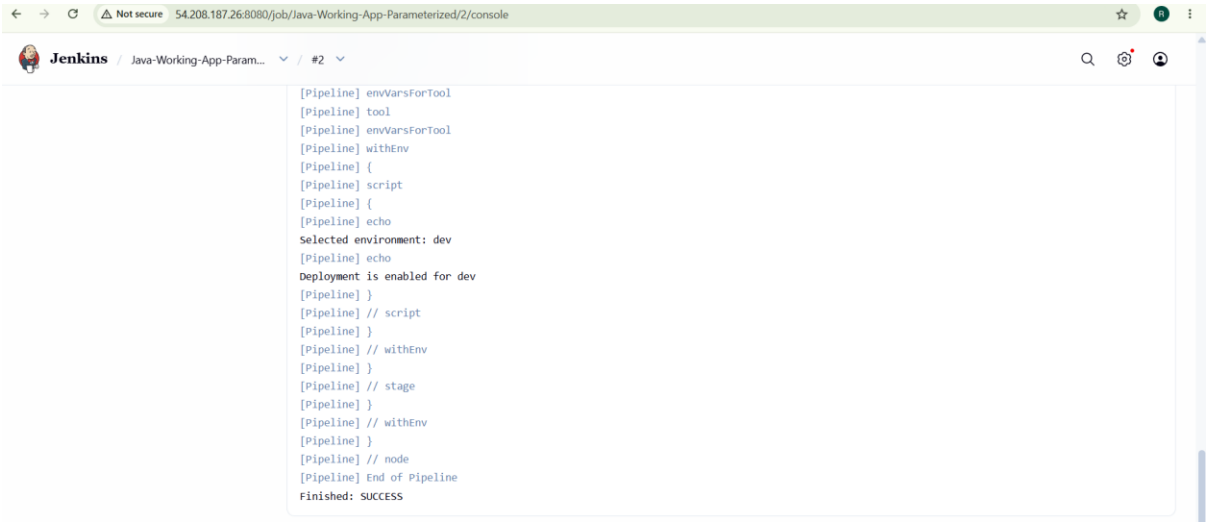





Not secure 54.208.187.26:8080/job/Java-Working-App-Parameterized/1/console
Jenkins / Java-Working-App-Param... / #1
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Selected environment: dev
[Pipeline] echo
Deployment skipped because DEPLOY is false
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 54.208.187.26:8080/job/Java-Working-App-Parameterized/build?delay=0sec
Jenkins / Java-Working-App-Param...
Status
Changes
Build with Parameters
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Builds
Filter
Today
#1 11:54
Pipeline Java-Working-App-Parameterized
This build requires parameters:
ENVIRONMENT
Select the deployment environment.
dev
DEPLOY
Enable or skip the deployment stage.
Build
Cancel


Not secure 54.208.187.26:8080/job/Java-Working-App-Parameterized/2/console
Jenkins / Java-Working-App-Param... / #2
Status
Changes
Console Output
Delete build '#2'
Parameters
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Download
Copy
View as plain text
Started by user Poojith R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Java-Working-App-Parameterized
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] git
The recommended git tool is: NONE



6. What are the global variables in Jenkins?

Jenkins provides built-in variables that let a job access information about its execution without manually entering that information.

For example, instead of manually typing the build number, Jenkins can provide the current build number through `BUILD_NUMBER`.

Some commonly used variables are:

Variable	Meaning
`BUILD_NUMBER`	Current build number
`JOB_NAME`	Name of the Jenkins job
`BUILD_ID`	Identifier for the current build
`BUILD_URL`	URL of the current build
`WORKSPACE`	Directory where Jenkins runs the job
`JENKINS_URL`	Base URL of the Jenkins server, if configured
`JENKINS_HOME`	Directory where Jenkins stores its configuration and data
`NODE_NAME`	Name of the Jenkins node executing the job